

รายงานสรุป Mini-DB Project

ระบบบริหารจัดการผู้ป่วยโรงพยาบาล (Hospital Management System)

KILLERUNAI HOSPITAL

เสนอ

อาจารย์ผู้สอน : รศ.ดร.อานนท์ รุ่งสว่าง

รายงานเล่มนี้เป็นส่วนหนึ่งของการเรียนรายวิชาระบบฐานข้อมูล หรัสวิชา 310-2209

ภาคการศึกษาที่ 1 เทอม 2 ปีการศึกษา 2569

สถาบันเทคโนโลยีจิตลดา

KILLERUNAI
HOSPITAL



ลำดับ	รหัสนักศึกษา	ชื่อ-นามสกุล
1	6810301001	นาย ธีรวัฒน์ ภาสพงษ์ไพศาล
2	6810301022	นาย ปริศร บุญเคลิม
3	6810301023	นาย รัฐภูมิ สุทธยาคม
4	6810301030	นาย ธารา งาสง่า
5	6810301034	นาย ฐิติพงศ์ โพธิศรี
6	6850301003	นาย สุวิจักขณ์ ไพศาลภาณุมาศ

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญของปัญหา

โรงพยาบาลขนาดกลางแห่งหนึ่งยังคงบริหารจัดการข้อมูลผู้ป่วย การนัดหมายและการจ่ายยาด้วยระบบเอกสารและการจดบันทึกแบบแยกส่วน ส่งผลให้เกิดปัญหาหลายด้าน ได้แก่ การจัดคิวตรวจที่ไม่เป็นระบบ การจ่ายยาที่ไม่มีการตรวจสอบสต็อกคงเหลือแบบเรียลไทม์ การคำนวณค่าปรับกรณีชำระเงินล่าช้าที่ต้องทำด้วยมือและเสี่ยงต่อความผิดพลาด การไม่มีระบบติดตามผู้ป่วยที่ค้างชำระเกินกำหนดอย่างเป็นระบบ (Blacklist) รวมถึงไม่มีการแจ้งเตือนเมื่อยาใกล้หมดอายุหรือสต็อกต่ำกว่าจุดสั่งซื้อ

คณะผู้จัดทำจึงออกแบบและพัฒนาฐานข้อมูลเชิงสัมพันธ์ (Relational Database) ร่วมกับระบบซอฟต์แวร์ เพื่อแก้ไขปัญหาดังกล่าวอย่างเป็นระบบ โดยอาศัยความรู้จากรายวิชาฐานข้อมูลในการออกแบบตาราง ความสัมพันธ์ ข้อจำกัดของข้อมูล (Constraints) และการเขียนคำสั่ง SQL เพื่อประมวลผลตรรกะทางธุรกิจ (Business Logic) ที่ซับซ้อน เช่น การคำนวณค่าปรับและการจัดการ Blacklist โดยอัตโนมัติ

1.2 วัตถุประสงค์ของโครงการ

- เพื่อออกแบบฐานข้อมูลเชิงสัมพันธ์ที่สะท้อนความสัมพันธ์ของข้อมูลในระบบโรงพยาบาลได้อย่างถูกต้องตามหลักการ Normalization
- เพื่อนำความรู้เรื่อง SQL (DDL, DML, Stored Procedure, Join, Aggregate Function) มาประยุกต์ใช้แก้ปัญหาจริง
- เพื่อพัฒนาระบบต้นแบบที่สามารถบริหารจัดการคิวตรวจ การจ่ายยา ค่าปรับ Blacklist และคลังยาได้แบบอัตโนมัติ

1.3 ขอบเขตของระบบ

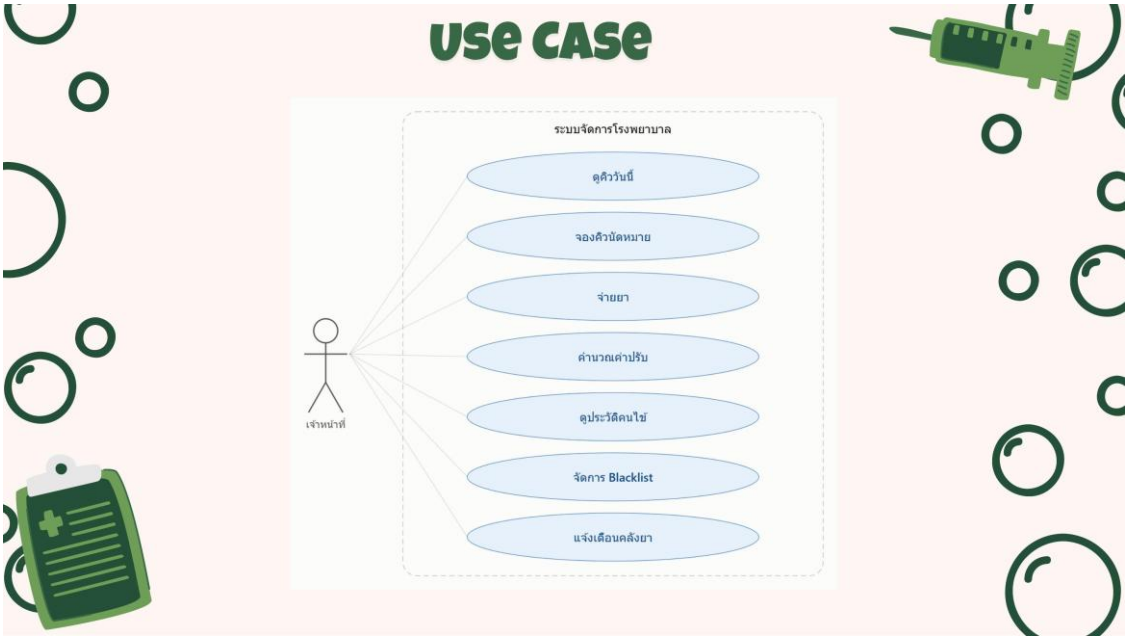
ระบบครอบคลุมการทำงาน 6 ส่วนหลัก ได้แก่ ระบบคิวตรวจวันนี้ ระบบจองคิว ระบบจ่ายยา ระบบคำนวณค่าปรับ ระบบ Blacklist ผู้ป่วย และระบบแจ้งเตือนคลังยา โดยพัฒนาฐานข้อมูลด้วย Microsoft SQL Server ส่วน Backend พัฒนาด้วย NestJS (Node.js) และส่วน Frontend พัฒนาด้วย React

บทที่ 2 การวิเคราะห์ระบบ

2.1 Use Case Diagram

ผู้ใช้งานหลักของระบบคือ "เจ้าหน้าที่" ซึ่งสามารถทำรายการต่าง ๆ ได้ 7 กรณีการใช้งาน (Use Case)

ดังแสดงในแผนภาพด้านล่าง



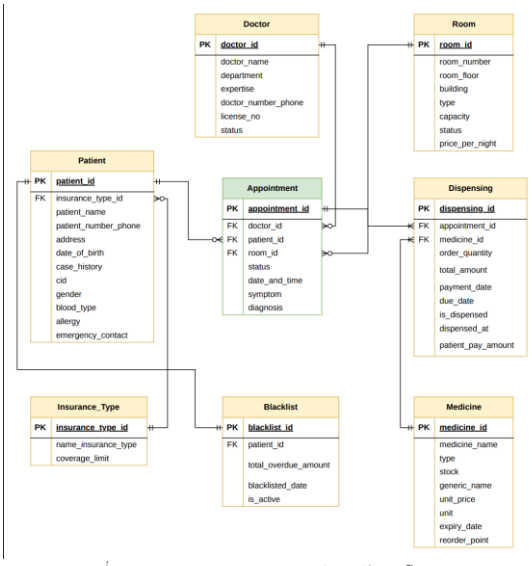
2.2 รายละเอียด Use Case

Use Case	คำอธิบาย
ดูคิววันนี้	แสดงรายการนัดหมายของวันปัจจุบันแบบเรียลไทม์
จองคิวนัดหมาย	บันทึกนัดหมายใหม่พร้อมอาการเบื้องต้น กรองเฉพาะแพทย์/ห้องที่พร้อมให้บริการ
จ่ายยา	ตัดสต็อกยาอัตโนมัติและคำนวณยอดที่ผู้ป่วยต้องชำระเองตามสิทธิ
คำนวณค่าปรับ	คำนวณค่าปรับกรณีชำระเงินล่าช้าตามสูตรที่กำหนดโดยอัตโนมัติ
ดูประวัติคนไข้	แสดงประวัติการรักษาและการรับยาของผู้ป่วยแต่ละราย
จัดการ Blacklist	ติดตามและปลดสถานะผู้ป่วยที่ค้างชำระเกินกำหนด
แจ้งเตือนคลังยา	แจ้งเตือนเมื่อยาใกล้หมดอายุหรือสต็อกต่ำกว่าจุดสั่งซื้อ

บทที่ 3 การออกแบบฐานข้อมูล

3.1 แผนภาพ ER Diagram

ฐานข้อมูลประกอบด้วย 8 ตาราง ได้แก่ Patient, Doctor, Room, Medicine, Appointment, Dispensing, Insurance_Type และ Blacklist โดยมีความสัมพันธ์แบบ One-to-Many (1:N) ทุกคู่ความสัมพันธ์ ดังแสดงในภาพต่อไปนี้



ภาพที่ 3.1 ER Diagram ของระบบบริหารจัดการโรงพยาบาล

3.3 ความสัมพันธ์ระหว่างตาราง (Relationships)

แผนภาพความสัมพันธ์ระหว่างตารางในระบบบริหารจัดการโรงพยาบาลแสดงโครงสร้างฐานข้อมูลแบบ Relational Database โดยตาราง Patient มีความสัมพันธ์แบบ One-to-Many กับตาราง Insurance_Type ตามสิทธิการรักษาพยาบาล และเชื่อมโยงแบบ One-to-Many กับตาราง Blacklist สำหรับตรวจสอบประวัติการค้างชำระเงิน ในส่วนของตารางศูนย์กลางอย่าง Appointment (การนัดหมาย) จะมีความสัมพันธ์แบบ Many-to-One เชื่อมโยงไปยังตารางหลัก ได้แก่ Doctor (แพทย์), Patient (คนไข้), และ Room (ห้องพัก) นอกจากนี้ตาราง Dispensing (การจ่ายยา) ยังมีความสัมพันธ์แบบ Many-to-One เชื่อมต่อไปยังตาราง Appointment และตาราง Medicine (คลังยา) เพื่อบันทึกรายละเอียดการจ่ายยาและคำนวณค่าใช้จ่ายได้อย่างครบถ้วนสมบูรณ์

อธิบายเพิ่มเติม

- คนไข้หนึ่งคนนัดหมายได้หลายครั้งหรือไม่มีก็ได้ แต่การนัดหมายหนึ่งครั้งเป็นของคนไข้แค่คนเดียว
- หมอหนึ่งคนตรวจได้หลายนัดหรือไม่มีก็ได้ แต่การนัดหมายหนึ่งครั้งมีหมอตรวจแค่คนเดียว
- ห้องหนึ่งห้องถูกใช้ในหลายนัดหมายหรือไม่มีก็ได้ แต่การนัดหมายหนึ่งครั้งใช้ห้องแค่ห้องเดียว
- การนัดหมายหนึ่งครั้งสั่งยาได้อย่างน้อย 1 รายการ แต่รายการยาที่สั่งหนึ่งรายการผูกกับการนัดหมายแค่ครั้งเดียว
- ยาหนึ่งชนิดถูกสั่งจ่ายได้หลายครั้งหรือไม่มีก็ได้ แต่รายการยาที่สั่งหนึ่งรายการระบุยาแค่ชนิดเดียว
- สิทธิการรักษาหนึ่งประเภทมีคนไข้ถือได้หลายคนหรือไม่มีก็ได้ แต่คนไข้หนึ่งคนถือสิทธิได้แค่ประเภทเดียว
- ผู้ป่วย 1 คน สามารถมีรายการในแบล็กลิสต์ได้สูงสุด 1 รายการ และรายการแบล็กลิสต์แต่ละรายการจะระบุถึงผู้ป่วยได้เพียง 1 คนเท่านั้น

บทที่ 4 อธิบายโค้ด SQL

อธิบายโครงสร้างและชุดคำสั่ง SQL ทั้ง 4 ส่วนที่ใช้ในการพัฒนาระบบ เริ่มตั้งแต่การสร้างฐานข้อมูลโครงสร้างหลัก การเพิ่มตารางและ Stored Procedure สำหรับจัดการระบบ ไปจนถึงการเพิ่มข้อมูลหลักและข้อมูลทดสอบ เพื่อให้ระบบบริหารจัดการโรงพยาบาลสามารถทำงานได้อย่างสมบูรณ์

ส่วนที่ 1 Create Database

```
IF DB_ID(N'HospitalDB') IS NULL
    CREATE DATABASE HospitalDB;
GO

USE HospitalDB;
GO

-- 1. Patient (ไม่มี FK)
IF OBJECT_ID(N'Patient', N'U') IS NULL
    CREATE TABLE Patient (
        Patient_ID INT IDENTITY(1,1) PRIMARY KEY,
        Name NVARCHAR(100) NOT NULL,
        Number_Phone VARCHAR(15),
        address NVARCHAR(255),
        Date_of_Birth DATE,
        Case_History NVARCHAR(MAX)
    );
GO
```

รูปภาพส่วนที่ 1.1 Create Database

หลักการทำงานของรูปนี้ คือ มันจะตรวจสอบก่อนว่า มีฐานข้อมูล HospitalDB ถ้าไม่มีมันจะสร้างฐานข้อมูลมาให้โดยอัตโนมัติ จากนั้นจึงเลือกใช้งานฐานข้อมูลที่สร้างและตรวจสอบการมีอยู่ของตาราง Patient หากยังไม่เจอตาราง จะทำการสร้างตารางเก็บข้อมูลคนไข้ไว้ พร้อมกำหนดคอลัมน์สำคัญและตั้งค่า Patient_ID เป็นคีย์หลัก (Primary Key) การเขียนเงื่อนไขตรวจสอบก่อนสร้างนั้น ทำให้สามารถรันสคริปต์ซ้ำได้อย่างปลอดภัยโดยไม่เกิดข้อผิดพลาด

*** ส่วนคำสั่ง Go ที่ อยู่นอกเหนือจากการเรียน มันคือ ตัวคั่นชุดคำสั่งเพื่อส่งไปยัง SQL Server ให้ประมวลผลคำสั่งของตัวมันเอง แล้วค่อยขึ้นประมวลผล อันใหม่ หรือก็คือแบ่งเป็นท่อนๆให้เข้าใจ ***

```sql
-- 2. Doctor (ไม่มี FK)
IF OBJECT_ID(N'Doctor', N'U') IS NULL
    CREATE TABLE Doctor (
        Doctor_ID INT IDENTITY(1,1) PRIMARY KEY,
        Name NVARCHAR(100) NOT NULL,
        department NVARCHAR(100),
        Expertise NVARCHAR(100)
    );
GO

-- 3. Room (ไม่มี FK)
IF OBJECT_ID(N'Room', N'U') IS NULL
    CREATE TABLE Room (
        Room_ID INT IDENTITY(1,1) PRIMARY KEY,
        Room_number VARCHAR(10) NOT NULL,
        Room_floor INT,
        building NVARCHAR(50),
        Type NVARCHAR(50),
        Capacity INT
    );
GO

-- 4. Medicine (ไม่มี FK)
IF OBJECT_ID(N'Medicine', N'U') IS NULL
    CREATE TABLE Medicine (
        Medicine_ID INT IDENTITY(1,1) PRIMARY KEY,
        Name NVARCHAR(100) NOT NULL,
        Type NVARCHAR(50),
        Stock INT NOT NULL DEFAULT 0
    );
GO
```

รูปภาพส่วนที่ 1.2 Create Doctor / Room / Medicine

หลักการทำงานของรูปนี้ คือ ใช้โค้ดชุดนี้ในการรันคำสั่ง IF OBJECT_ID เพื่อตรวจสอบและสร้างตารางหลักที่ยังไม่มีความสัมพันธ์แบบ Foreign Key (FK) จำนวน 3 ตาราง ได้แก่ ตารางหมอ (Doctor), ตารางห้องพักผู้ป่วย (Room), และตารางคลังยา (Medicine) พร้อมทั้งกำหนดคอลัมน์สำคัญและตั้งค่ารหัสประจำตัวของแต่ละตารางเป็น Primary Key แบบเพิ่มค่าอัตโนมัติ (IDENTITY) รวมทั้งกำหนดค่าเริ่มต้นให้กับจำนวนสต๊อกยา เพื่อให้ฐานข้อมูลมีความพร้อมก่อนเชื่อมโยงความสัมพันธ์ในตารางอื่น *** ในส่วนของคำสั่ง IF OBJECT_ID ใช้สำหรับ **ตรวจสอบว่ามีตารางในฐานข้อมูลอยู่แล้วหรือไม่** ถ้ายังไม่มี (IS NULL) ค่อยทำการสร้างตารางใหม่ เพื่อป้องกันไม่ให้เกิด Error เมื่อกดรันระบบ***

```sql
-- 5. Appointment (มี FK ไปหา Patient, Doctor, Room)
IF OBJECT_ID(N'Appointment', N'U') IS NULL
    CREATE TABLE Appointment (
        Appointment_ID INT IDENTITY(1,1) PRIMARY KEY,
        Doctor_ID INT NOT NULL,
        Patient_ID INT NOT NULL,
        Room_ID INT NOT NULL,
        Status NVARCHAR(20) NOT NULL,
        Date_and_Time DATETIME NOT NULL,
        FOREIGN KEY (Doctor_ID) REFERENCES Doctor(Doctor_ID),
        FOREIGN KEY (Patient_ID) REFERENCES Patient(Patient_ID),
        FOREIGN KEY (Room_ID) REFERENCES Room(Room_ID)
    );
GO

-- 6. Dispensing (มี FK ไปหา Appointment, Medicine)
IF OBJECT_ID(N'Dispensing', N'U') IS NULL
    CREATE TABLE Dispensing (
        Dispensing_ID INT IDENTITY(1,1) PRIMARY KEY,
        Appointment_ID INT NOT NULL,
        Medicine_ID INT NOT NULL,
        Order_quantity INT NOT NULL,
        Total_amount DECIMAL(10,2) NOT NULL,
        Payment_date DATE,
        Due_date DATE,
        FOREIGN KEY (Appointment_ID) REFERENCES Appointment(Appointment_ID),
        FOREIGN KEY (Medicine_ID) REFERENCES Medicine(Medicine_ID)
    );
GO
```

รูปภาพส่วนที่ 1.3 Create Appointment / Dispensing

หลักการทำงานของรูปนี้ คือ ใช้สำหรับสร้างตารางที่มีความสัมพันธ์แบบ Foreign Key (FK) เชื่อมโยงไปยังตารางอื่น ได้แก่ ตารางการนัดหมาย (Appointment) ที่เชื่อมไปยังหมอ คนไข้ และห้องพักผู้ป่วย รวมถึงตารางการจ่ายยา (Dispensing) ที่เชื่อมโยงกับข้อมูลการนัดหมายและคลังยา พร้อมทั้งกำหนดคอลัมน์สำคัญ เช่น จำนวนยา ยอดเงินรวม วันที่ชำระเงิน และวันครบกำหนดชำระ เพื่อให้ฐานข้อมูลมีความสัมพันธ์เชิงตรรกะที่สมบูรณ์สำหรับการบริหารจัดการโรงพยาบาล

ส่วนที่ 2 Update

```sql
USE HospitalDB;
GO

IF OBJECT_ID('Insurance_Type', 'U') IS NULL
    CREATE TABLE Insurance_Type (
        Insurance_Type_ID INT IDENTITY(1,1) PRIMARY KEY,
        Name NVARCHAR(50) NOT NULL,
        Coverage_Limit DECIMAL(10,2) NOT NULL DEFAULT 0
    );
GO

IF NOT EXISTS (SELECT 1 FROM Insurance_Type)
    INSERT INTO Insurance_Type (Name, Coverage_Limit) VALUES
    (N'บัตรทอง (สปสช.)', 500.00),
    (N'ประกันสังคม',      800.00),
    (N'ข้าราชการ',        1500.00),
    (N'ชำระเงินเอง',      0.00);


IF OBJECT_ID('tempdb..#dup_insurance_map') IS NOT NULL DROP TABLE #dup_insurance_map;
SELECT it.Insurance_Type_ID AS Old_ID, keep.Keep_ID
INTO #dup_insurance_map
FROM Insurance_Type it
JOIN (
    SELECT Name, MIN(Insurance_Type_ID) AS Keep_ID
    FROM Insurance_Type
    GROUP BY Name
) keep ON it.Name = keep.Name
WHERE it.Insurance_Type_ID <> keep.Keep_ID;
```

รูปภาพส่วนที่ 2.1 สร้างตาราง Insurance_Type

หลักการทำงานของรูปนี้ คือ การสร้างตาราง Insurance_Type สำหรับจัดเก็บข้อมูลประเภทสิทธิการรักษาพยาบาลและวงเงินคุ้มครอง พร้อมตรวจสอบไม่ให้เกิดการสร้างซ้ำ จากนั้นทำการเพิ่มข้อมูลตั้งต้นของสิทธิการรักษา เช่น บัตรทอง ประกันสังคม ข้าราชการ และชำระเงินเองเข้าสู่ระบบ พร้อมทั้งมีการตรวจสอบและจัดการข้อมูลที่อาจซ้ำซ้อนผ่านตารางชั่วคราว เพื่อความเป็นระเบียบและความถูกต้องแม่นยำของฐานข้อมูล

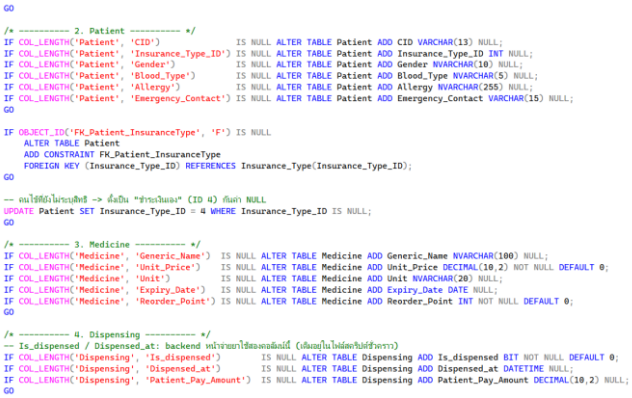
```sql
UPDATE p
SET p.Insurance_Type_ID = m.Keep_ID
FROM Patient p
JOIN #dup_insurance_map m ON p.Insurance_Type_ID = m.Old_ID;

DELETE it
FROM Insurance_Type it
JOIN #dup_insurance_map m ON it.Insurance_Type_ID = m.Old_ID;

DROP TABLE #dup_insurance_map;
GO
```

รูปภาพส่วนที่ 2.2 Update ข้อมูล สิทธิ

หลักการทำงานของรูปนี้ คือ จัดการและทำความสะอาดข้อมูลสิทธิการรักษาที่ซ้ำซ้อน โดยการอัปเดตข้อมูลรหัสสิทธิในตาราง Patient ให้เชื่อมโยงไปยังรหัสหลักที่ถูกต้อง จากนั้นทำการลบรายการข้อมูลสิทธิที่ซ้ำซ้อนออกจากตารางหลัก Insurance_Type ก่อนที่จะลบตารางชั่วคราวทิ้งเพื่อเคลียร์พื้นที่และทำให้ข้อมูลในระบบมีความถูกต้องเป็นระเบียบ



```
GO

/* ---------- 2. Patient ---------- */
IF COL_LENGTH('Patient', 'CID')              IS NULL ALTER TABLE Patient ADD CID VARCHAR(13) NULL;
IF COL_LENGTH('Patient', 'Insurance_Type_ID') IS NULL ALTER TABLE Patient ADD Insurance_Type_ID INT NULL;
IF COL_LENGTH('Patient', 'Gender')           IS NULL ALTER TABLE Patient ADD Gender NVARCHAR(10) NULL;
IF COL_LENGTH('Patient', 'Blood_Type')       IS NULL ALTER TABLE Patient ADD Blood_Type NVARCHAR(5) NULL;
IF COL_LENGTH('Patient', 'Allergy')          IS NULL ALTER TABLE Patient ADD Allergy NVARCHAR(255) NULL;
IF COL_LENGTH('Patient', 'Emergency_Contact') IS NULL ALTER TABLE Patient ADD Emergency_Contact VARCHAR(15) NULL;
GO

IF OBJECT_ID('FK_Patient_InsuranceType', 'F') IS NULL
    ALTER TABLE Patient
    ADD CONSTRAINT FK_Patient_InsuranceType
    FOREIGN KEY (Insurance_Type_ID) REFERENCES Insurance_Type(Insurance_Type_ID);
GO

-- คนไข้ที่ยังไม่ระบุสิทธิ -> ตั้งเป็น "ชำระเงินเอง" (ID 4) ก็แค่ NULL
UPDATE Patient SET Insurance_Type_ID = 4 WHERE Insurance_Type_ID IS NULL;
GO

/* ---------- 3. Medicine ---------- */
IF COL_LENGTH('Medicine', 'Generic_Name')  IS NULL ALTER TABLE Medicine ADD Generic_Name NVARCHAR(100) NULL;
IF COL_LENGTH('Medicine', 'Unit_Price')    IS NULL ALTER TABLE Medicine ADD Unit_Price DECIMAL(10,2) NOT NULL DEFAULT 0;
IF COL_LENGTH('Medicine', 'Unit')          IS NULL ALTER TABLE Medicine ADD Unit NVARCHAR(20) NULL;
IF COL_LENGTH('Medicine', 'Expiry_Date')   IS NULL ALTER TABLE Medicine ADD Expiry_Date DATE NULL;
IF COL_LENGTH('Medicine', 'Reorder_Point') IS NULL ALTER TABLE Medicine ADD Reorder_Point INT NOT NULL DEFAULT 0;
GO

/* ---------- 4. Dispensing ---------- */
-- Is_dispensed / Dispensed_at: backend หน้าจ่ายยาใช้ตอนกดยืนยัน (เติมอยู่ในไฟล์ตรงนี้ด้วยก็ได้ชัวร์ๆ)
IF COL_LENGTH('Dispensing', 'Is_dispensed')       IS NULL ALTER TABLE Dispensing ADD Is_dispensed BIT NOT NULL DEFAULT 0;
IF COL_LENGTH('Dispensing', 'Dispensed_at')       IS NULL ALTER TABLE Dispensing ADD Dispensed_at DATETIME NULL;
IF COL_LENGTH('Dispensing', 'Patient_Pay_Amount') IS NULL ALTER TABLE Dispensing ADD Patient_Pay_Amount DECIMAL(10,2) NULL;
GO
```

รูปภาพส่วนที่ 2.2 Update ตาราง โดยคำสั่ง COL_LENGTH

หลักการทำงานของรูปนี้ คือ อัปเดตและปรับปรุงโครงสร้างตารางเดิมโดยใช้คำสั่ง COL_LENGTH เพื่อตรวจสอบและเพิ่มคอลัมน์ใหม่ที่จำเป็น เช่น ข้อมูลส่วนตัวของคนไข้ ข้อมูลยา และสถานะการจ่ายยา พร้อมทั้งสร้างความสัมพันธ์แบบ Foreign Key เชื่อมโยงระหว่างตารางคนไข้กับประเภทสิทธิการรักษา รวมถึงมีการกำหนดค่าเริ่มต้นให้กับคนไข้ที่ยังไม่ได้ระบุสิทธิให้เป็นแบบชำระเงินเองโดยอัตโนมัติ

คำสั่ง COL_LENGTH คือ **คำสั่งตรวจสอบว่ามีคอลัมน์นั้นอยู่ในตารางแล้วหรือยัง** โดยถ้าไม่พบ ก็จะได้ค่าเป็น NULL ระบบจะทำการเพิ่มคอลัมน์ใหม่เข้าไป เพื่อป้องกันไม่ให้เกิดข้อผิดพลาดเวลาที่รันสคริปต์ซ้ำ

```
-- คำนวณยอดที่คนไข้ต้องจ่ายเอง (ราคา - วงเงินสิทธิ, ไม่ต่ำกว่า 0) ให้แถวที่ยังไม่มีค่า
UPDATE d
SET d.Patient_Pay_Amount =
    CASE
        WHEN d.Total_amount - ISNULL(it.Coverage_Limit, 0) > 0
        THEN d.Total_amount - ISNULL(it.Coverage_Limit, 0)
        ELSE 0
    END
FROM Dispensing d
JOIN Appointment a ON d.Appointment_ID = a.Appointment_ID
JOIN Patient p ON a.Patient_ID = p.Patient_ID
LEFT JOIN Insurance_Type it ON p.Insurance_Type_ID = it.Insurance_Type_ID
WHERE d.Patient_Pay_Amount IS NULL;
GO
```

รูปภาพส่วนที่ 2.3 คำนวณยอดที่คนไข้ต้องจ่ายเอง

โค้ดชุดนี้ทำหน้าที่คำนวณยอดเงินที่คนไข้ต้องชำระเอง โดยนำราคารวมหักลบด้วยวงเงินคุ้มครองตามสิทธิการรักษาพยาบาล และกำหนดเงื่อนไขไม่ให้ยอดเงินติดลบ (ค่าต่ำสุดคือ 0 บาท) สำหรับแถวข้อมูลที่ยังไม่มีการคำนวณ พร้อมทั้งเชื่อมโยงข้อมูลระหว่างตารางการจ่ายยา การนัดหมาย คนไข้ และประเภทสิทธิเข้าด้วยกันอย่างแม่นยำ

```
/* ---------- 5. Doctor ---------- */
IF COL_LENGTH('Doctor', 'Number_Phone') IS NULL ALTER TABLE Doctor ADD Number_Phone VARCHAR(15) NULL;
IF COL_LENGTH('Doctor', 'License_No')   IS NULL ALTER TABLE Doctor ADD License_No NVARCHAR(20) NULL;
IF COL_LENGTH('Doctor', 'Status')       IS NULL ALTER TABLE Doctor ADD Status NVARCHAR(20) NOT NULL DEFAULT N'ปฏิบัติงาน';
GO

-- เลขใบ ว. มีตัวอักษรไทย ต้องเป็น NVARCHAR (ถ้าเป็น VARCHAR ตัว ว. จะกลายเป็น ?)
ALTER TABLE Doctor ALTER COLUMN License_No NVARCHAR(20) NULL;
GO

/* ---------- 6. Room ---------- */
IF COL_LENGTH('Room', 'Type')            IS NULL ALTER TABLE Room ADD Type NVARCHAR(50) NULL;    -- เผื่อ DB ที่สร้างจากไฟล์ 1 เวอร์ชันเก่า
IF COL_LENGTH('Room', 'Capacity')        IS NULL ALTER TABLE Room ADD Capacity INT NULL;
IF COL_LENGTH('Room', 'Status')          IS NULL ALTER TABLE Room ADD Status NVARCHAR(20) NOT NULL DEFAULT N'ว่าง';
IF COL_LENGTH('Room', 'Price_per_night') IS NULL ALTER TABLE Room ADD Price_per_night DECIMAL(10,2) NULL;
GO

/* ---------- 7. Appointment ---------- */
IF COL_LENGTH('Appointment', 'Symptom')   IS NULL ALTER TABLE Appointment ADD Symptom NVARCHAR(255) NULL;
IF COL_LENGTH('Appointment', 'Diagnosis') IS NULL ALTER TABLE Appointment ADD Diagnosis NVARCHAR(255) NULL;
GO

/* ---------- 8. Blacklist ---------- */
IF OBJECT_ID('Blacklist', 'U') IS NULL
    CREATE TABLE Blacklist (
        Blacklist_ID INT IDENTITY(1,1) PRIMARY KEY,
        Patient_ID INT NOT NULL,
        Total_overdue_amount DECIMAL(10,2) NOT NULL DEFAULT 0,
        Blacklisted_date DATETIME NOT NULL DEFAULT GETDATE(),
        Is_active BIT NOT NULL DEFAULT 1,
        FOREIGN KEY (Patient_ID) REFERENCES Patient(Patient_ID)
    );
GO
```

รูปภาพส่วนที่ 2.4 ตรวจสอบและเพิ่มคอลัมน์ใหม่

โค้ดชุดนี้ใช้คำสั่ง COL_LENGTH เพื่อตรวจสอบและเพิ่มคอลัมน์ใหม่ที่จำเป็นสำหรับตารางแพทย์ ห้องพัก และการนัดหมาย เช่น เบอร์โทร เลขใบประกอบวิชาชีพ สถานะห้อง และประวัติอาการป่วย พร้อมทั้งปรับปรุงชนิดข้อมูลของฟิลด์ใบประกอบวิชาชีพให้รองรับตัวอักษรภาษาไทยอย่างถูกต้อง รวมถึงมีการสร้างตาราง Blacklist ใหม่พร้อมกำหนดคีย์หลักและฟอเรนคีย์เชื่อมโยงไปยังตารางคนไข้ สำหรับบันทึกข้อมูลและสถานะการค้างชำระเงินในระบบ

```
CREATE OR ALTER PROCEDURE sp_Update_Blacklist
    @OverdueDays INT = 30
AS
BEGIN
    SET NOCOUNT ON;

    IF OBJECT_ID('tempdb..#Overdue') IS NOT NULL DROP TABLE #Overdue;

    SELECT a.Patient_ID,
           SUM(ISNULL(d.Patient_Pay_Amount, d.Total_amount)) AS Amount
    INTO #Overdue
    FROM Dispensing d
    JOIN Appointment a ON d.Appointment_ID = a.Appointment_ID
    WHERE d.Is_dispensed = 1
      AND d.Payment_date IS NULL
      AND DATEDIFF(DAY, d.Due_date, GETDATE()) > @OverdueDays
      AND ISNULL(d.Patient_Pay_Amount, d.Total_amount) > 0
    GROUP BY a.Patient_ID;
```

รูปภาพส่วนที่ 2.5 คัดกรองและรวมยอดหนี้ของคนไข้

โค้ดชุดนี้เป็นการสร้าง Stored Procedure ในชื่อ sp_Update_Blacklist เพื่อทำหน้าที่คัดกรองและรวมยอดหนี้ของคนไข้ที่รับยาไปแล้วแต่ค้างชำระเงินเกินเวลาที่กำหนด โดยระบบจะดึงข้อมูลที่ตรงตามเงื่อนไขมาเก็บบันทึกไว้ในตารางชั่วคราวตรง Overdue เพื่อเตรียมนำไปประมวลผลอัปเดตสถานะ Blacklist ของคนไข้ในขั้นตอนต่อไปอย่างเป็นระบบ

```
    -- 1) เพิ่มเข้า Blacklist
    INSERT INTO Blacklist (Patient_ID, Total_overdue_amount, Blacklisted_date, Is_active)
    SELECT o.Patient_ID, o.Amount, GETDATE(), 1
    FROM #Overdue o
    WHERE NOT EXISTS (
        SELECT 1 FROM Blacklist b
        WHERE b.Patient_ID = o.Patient_ID AND b.Is_active = 1
    );

    -- 2) อัปเดตยอดค้างล่าสุดของคนที่ยังอยู่ใน Blacklist
    UPDATE b
    SET b.Total_overdue_amount = o.Amount
    FROM Blacklist b
    JOIN #Overdue o ON o.Patient_ID = b.Patient_ID
    WHERE b.Is_active = 1
      AND b.Total_overdue_amount <> o.Amount;

    -- 3) ปลดแบนคนที่ไม่มีรายการค้างเข้าเงื่อนไขแล้ว
    UPDATE b
    SET b.Is_active = 0
    FROM Blacklist b
    WHERE b.Is_active = 1
      AND NOT EXISTS (SELECT 1 FROM #Overdue o WHERE o.Patient_ID = b.Patient_ID);

    DROP TABLE #Overdue;
END
GO

/* ---------- เช็คผลลัพธ์ ---------- */
SELECT * FROM Insurance_Type;
SELECT Patient_ID, Name, CID, Insurance_Type_ID FROM Patient;
```

รูปภาพส่วนที่ 2.6 คัดกรองและรวมยอดหนี้ของคนไข้

โค้ดชุดนี้ดำเนินการต่อยอดภายใน Stored Procedure โดยแบ่งเป็น 3 ขั้นตอนหลัก ได้แก่ การเพิ่มรายชื่อคนไข้ใหม่เข้าสู่ตาราง Blacklist, การอัปเดตยอดหนี้ที่ค้างล่าสุดสำหรับผู้ที่อยู่ในระบบ, และการปลดแบนสถานะให้กับคนไข้ที่ไม่มีรายการค้างชำระจ่ายแล้ว พร้อมทั้งลบตารางชั่วคราวและปิดท้ายด้วยชุดคำสั่งที่เอาไว้ตรวจสอบผลลัพธ์ของข้อมูลสิทธิการรักษาและข้อมูลคนไข้เพื่อความถูกต้องสมบูรณ์

ส่วนที่ 3

```sql
USE HospitalDB;
GO

IF OBJECT_ID('Blacklist','U') IS NOT NULL DELETE FROM Blacklist;
DELETE FROM Dispensing;
DELETE FROM Appointment;
DELETE FROM Patient;
DELETE FROM Doctor;
DELETE FROM Room;
DELETE FROM Medicine;
GO

DBCC CHECKIDENT ('Patient', RESEED, 0);
DBCC CHECKIDENT ('Doctor', RESEED, 0);
DBCC CHECKIDENT ('Room', RESEED, 0);
DBCC CHECKIDENT ('Medicine', RESEED, 0);
DBCC CHECKIDENT ('Appointment', RESEED, 0);
DBCC CHECKIDENT ('Dispensing', RESEED, 0);
IF OBJECT_ID('Blacklist','U') IS NOT NULL DBCC CHECKIDENT ('Blacklist', RESEED, 0);
GO

INSERT INTO Patient (Name, Number_Phone, address, Date_of_Birth, Case_History, CID, Insurance_Type_ID, Gender, Blood_Type, Allergy, Emergency_Contact) VALUES
(N'อุมิน เชลสเบียล' , '08042365897', N'กรุงเทพฯ'  , '2006-09-28', N'ไม่มี'  , '1849302761548', 2, N'หญิง', N'O' , N'ไม่มี', '0873491285'),
(N'ปีเตอร์ พาร์คเกอร์', '0928501146', N'สมุทรปราการ', '2009-12-14', N'ไม่มี'  , '3021958473610', 4, N'ชาย' , N'A' , N'ไม่มี', '09188230047'),
(N'ชญานนท์ วงษ์'  , '0812356890', N'ชลบุรี'     , '2000-01-04', N'ไม่มี'  , '5637291840572', 1, N'ชาย' , N'B' , N'ไม่มี', '0654290187'),
(N'คริส สนามแดง'  , '0964419256', N'เชียงใหม่'  , '1994-03-15', N'เบาหวาน', '2749183056291', 3, N'ชาย' , N'AB', N'ไม่มี', '0837105629'),
(N'วิเลียม อาร์มตอน' , '0895155210', N'สงขลา'   , '1987-11-17', N'ความดัน', '4903827165042', 4, N'ชาย' , N'O' , N'เพนิซิลลิน', '0761948520');
GO

INSERT INTO Doctor (Name, department, Expertise, Number_Phone, License_No, Status) VALUES
(N'ชากุระ ฮากุระ'  , N'อายุรกรรม'   , N'โรคหัวใจ'           , '0891112221', N'ว.11234', N'ปฏิบัติงาน'),
(N'คุจิกิ ลูเคีย'  , N'อายุรกรรม'   , N'กระดูกและข้อ'       , '0891112222', N'ว.11235', N'ปฏิบัติงาน'),
(N'ไมเคิล คาร์ริก' , N'ศัลยกรรม'    , N'ศัลยแพทย์ระบบประสาท', '0891112223', N'ว.11236', N'ปฏิบัติงาน'),
(N'วินนี เดอะพูห์', N'กุมารเวชกรรม', N'รักษาโรคในเด็ก'     , '0891112224', N'ว.11237', N'ลา');
GO

INSERT INTO Room (Room_number, Room_floor, building, Type, Capacity, Status, Price_per_night) VALUES
('2010', 2, 1, N'ห้องรวม'   , 20, N'ว่าง',         500.00),
('2026', 2, 1, N'ห้องรวม'   , 20, N'ว่าง',         500.00),
('3001', 3, 1, N'ห้องพิเศษ' , 2 , N'ไม่ว่าง',     1500.00),
('3005', 3, 1, N'ห้องพิเศษ' , 2 , N'ว่าง',        1500.00),
('4100', 4, 2, N'ห้อง ICU'  , 10, N'ว่าง',        3000.00),
('5300', 5, 2, N'ห้องผ่าตัด', 8 , N'ปิดปรับปรุง', 5000.00);
GO

INSERT INTO Medicine (Name, Type, Stock, Generic_Name, Unit_Price, Unit, Expiry_Date, Reorder_Point) VALUES
(N'พาราเซตามอล' , N'ยาแก้ปวดลดไข้', 500, N'Paracetamol', 2.00, N'เม็ด' , '2027-06-30', 50),
(N'อะม็อกซิลลิน', N'ยาปฏิชีวนะ'  , 300, N'Amoxicillin', 5.00, N'แคปซูล', '2027-03-31', 40),
(N'เซทิริซีน'  , N'ยาแก้แพ้'    , 200, N'Cetirizine' , 3.00, N'เม็ด' , '2027-08-31', 30),
(N'โอเมพราโซล', N'ยาลดกรดในกระเพาะ', 150, N'Omeprazole' , 6.00, N'แคปซูล', '2027-01-31', 20),
(N'ไอบูโพรเฟน', N'ยาแก้ปวดลดอักเสบ', 250, N'Ibuprofen' , 4.00, N'เม็ด' , '2027-05-31', 30);
GO

SELECT * FROM Patient;
SELECT * FROM Doctor;
SELECT * FROM Room;
SELECT * FROM Medicine;
```

ส่วนที่ 3.1 กำหนดค่า Default ของ Database

โค้ดชุดนี้เริ่มต้นการทำงานด้วยการเชื่อมต่อเข้าสู่ฐานข้อมูล HospitalDB และใช้คำสั่ง DELETE เพื่อเคลียร์ข้อมูลเก่าทั้งหมดออกจากตารางหลัก รวมถึงมีการใช้งานคำสั่ง DBCC CHECKIDENT เพื่อรีเซ็ตค่าลำดับไอดี (Identity) ของทุกตารางให้เริ่มต้นนับใหม่ที่ 1 อย่างเป็นระเบียบ จากนั้นจึงดำเนินการเพิ่มข้อมูลตั้งต้น (Master Data) ลงในตารางสำคัญของระบบ ไม่ว่าจะเป็นข้อมูลส่วนตัวของคนไข้ ข้อมูลประวัติและสังกัดของหมอ ข้อมูลห้องพักคนไข้ประเภทต่างๆ และรายการคลังยาพร้อมจำนวนสต็อกเริ่มต้น ขั้นตอนนี้ช่วยให้ฐานข้อมูลมีความพร้อมและมีข้อมูลพื้นฐานที่จำเป็นครบถ้วนสำหรับการจำลองการทำงานจริง พร้อมทั้งปิดท้ายด้วยคำสั่ง SELECT เพื่อดึงข้อมูลทั้งหมดจากแต่ละตารางขึ้นมาแสดงผล และตรวจสอบความถูกต้องแม่นยำก่อนนำสคริปต์ไปใช้ในขั้นตอนถัดไป

ส่วนที่ 4

```sql
USE HospitalDB;
GO

IF OBJECT_ID('Blacklist', 'U') IS NOT NULL DELETE FROM Blacklist;
DELETE FROM Dispensing;
DELETE FROM Appointment;
DBCC CHECKIDENT ('Appointment', RESEED, 0);
DBCC CHECKIDENT ('Dispensing', RESEED, 0);
IF OBJECT_ID('Blacklist', 'U') IS NOT NULL DBCC CHECKIDENT ('Blacklist', RESEED, 0);
GO

DECLARE @today DATE = CAST(GETDATE() AS DATE);

INSERT INTO Appointment (Doctor_ID, Patient_ID, Room_ID, Status, Date_and_Time, Symptom, Diagnosis) VALUES
(1, 1, 1, N'รอคิว',      DATEADD(HOUR,  9, CAST(@today AS DATETIME)),                    N'ใจสั่น เหนื่อยง่าย',   NULL),
(2, 2, 3, N'กำลังตรวจ', DATEADD(HOUR, 10, CAST(@today AS DATETIME)),                    N'ปวดหลังเรื้อรัง',      NULL),
(3, 3, 5, N'รอคิว',      DATEADD(HOUR, 14, CAST(@today AS DATETIME)),                    N'ปวดศีรษะบ่อย',         NULL),
(1, 4, 2, N'รอคิว',      DATEADD(HOUR, 16, CAST(@today AS DATETIME)),                    N'ไข้ ไอ 3 วัน',         NULL),
(3, 3, 5, N'เสร็จแล้ว', DATEADD(HOUR, 10, CAST(DATEADD(DAY,  -5, @today) AS DATETIME)), N'ปวดท้อง แสบร้อนกลางอก', N'กรดไหลย้อน'),
(2, 1, 1, N'เสร็จแล้ว', DATEADD(HOUR, 11, CAST(DATEADD(DAY, -10, @today) AS DATETIME)), N'ปวดเมื่อยกล้ามเนื้อ',   N'กล้ามเนื้ออักเสบ'),
(1, 5, 2, N'เสร็จแล้ว', DATEADD(HOUR, 13, CAST(DATEADD(DAY, -45, @today) AS DATETIME)), N'ไข้หวัด',              N'ไข้หวัดธรรมดา'),
(1, 5, 4, N'ยกเลิก',     DATEADD(HOUR, 14, CAST(DATEADD(DAY,  -8, @today) AS DATETIME)), N'ตรวจสุขภาพประจำปี',    NULL),
(1, 2, 1, N'เสร็จแล้ว', DATEADD(HOUR,  9, CAST(DATEADD(DAY,  -3, @today) AS DATETIME)), N'ผื่นคัน แพ้อากาศ',     N'โรคภูมิแพ้');

INSERT INTO Dispensing (Appointment_ID, Medicine_ID, Order_quantity, Total_amount, Payment_date, Due_date, Is_dispensed, Dispensed_at) VALUES

(5, 2, 30, 0.00, DATEADD(DAY, -2, @today),  DATEADD(DAY, -5,  @today), 1, DATEADD(DAY, -5, GETDATE())),
(5, 4, 28, 0.00, DATEADD(DAY, -1, @today),  DATEADD(DAY, -5,  @today), 1, DATEADD(DAY, -5, GETDATE())),
(6, 1, 20, 0.00, DATEADD(DAY,-10, @today),  DATEADD(DAY,-10,  @today), 1, DATEADD(DAY,-10, GETDATE())),

(7, 5, 20, 0.00, NULL,                      DATEADD(DAY,-40,  @today), 1, DATEADD(DAY,-45, GETDATE())),

(9, 3, 30, 0.00, NULL,                      DATEADD(DAY,  7,  @today), 0, NULL),
(9, 1, 20, 0.00, NULL,                      DATEADD(DAY,  7,  @today), 0, NULL),
(6, 4, 14, 0.00, NULL,                      DATEADD(DAY,  7,  @today), 0, NULL),
(5, 4, 100, 0.00, NULL,                      DATEADD(DAY,  7,  @today), 0, NULL);
GO

UPDATE d
SET d.Total_amount = m.Unit_Price * d.Order_quantity
FROM Dispensing d
JOIN Medicine m ON d.Medicine_ID = m.Medicine_ID;
GO

UPDATE d
SET d.Patient_Pay_Amount =
    CASE WHEN d.Total_amount - ISNULL(it.Coverage_Limit, 0) > 0
         THEN d.Total_amount - ISNULL(it.Coverage_Limit, 0) ELSE 0 END
FROM Dispensing d
JOIN Appointment a ON d.Appointment_ID = a.Appointment_ID
JOIN Patient p ON a.Patient_ID = p.Patient_ID
LEFT JOIN Insurance_Type it ON p.Insurance_Type_ID = it.Insurance_Type_ID;
GO

IF OBJECT_ID('sp_Update_Blacklist', 'P') IS NOT NULL EXEC sp_Update_Blacklist;
GO

SELECT a.Appointment_ID, p.Name, a.Status, a.Date_and_Time, a.Symptom, a.Diagnosis
FROM Appointment a JOIN Patient p ON a.Patient_ID = p.Patient_ID ORDER BY a.Appointment_ID;
SELECT * FROM Dispensing;
SELECT * FROM Blacklist;
```

ส่วนที่ 4.1 เพิ่มข้อมูลตัวอย่าง (Demo Data)

โค้ดชุดนี้เป็นการเพิ่มข้อมูลตัวอย่าง (Demo Data) สำหรับการนัดหมายและการจ่ายยาของโรงพยาบาล พร้อมทั้งใช้ฟังก์ชันจัดการวันที่เพื่อจำลองเหตุการณ์ทั้งในอดีตและในปัจจุบัน ต่อมาจะทำการคำนวณราคารวมของยาแต่ละรายการ และคำนวณยอดเงินที่คนไข้ต้องชำระเองโดยหักลบกับวงเงินสิทธิการรักษาพยาบาลโดยอัตโนมัติ พร้อมทั้งเรียกใช้งาน Stored Procedure ในชื่อ sp_Update_Blacklist เพื่อประมวลผลและอัปเดตสถานะการค้างชำระเงินของคนไข้เข้าสู่ระบบแบล็กลิสต์ ก่อนที่จะปิดท้ายด้วยชุดคำสั่ง SELECT เพื่อดึงข้อมูลผลลัพธ์ทั้งหมดของตารางการนัดหมาย การจ่ายยา และรายการแบล็กลิสต์ขึ้นมาแสดงผลสำหรับตรวจสอบความถูกต้องครบถ้วน

บทที่ 5 สรุปและข้อเสนอแนะ

51 สรุปผลการดำเนินงาน

คณะผู้จัดทำสามารถออกแบบและพัฒนาฐานข้อมูลเชิงสัมพันธ์สำหรับระบบบริหารจัดการโรงพยาบาลได้สำเร็จ ครบทั้ง 8 ตาราง พร้อมความสัมพันธ์และ Constraints ที่ถูกต้องตามหลักการ รวมถึงพัฒนา Stored Procedure สำหรับจัดการ Blacklist โดยอัตโนมัติ และเชื่อมต่อกับระบบ Backend/Frontend ได้สำเร็จ ระบบสามารถทำงานได้ครบทั้ง 6 ฟีเจอร์หลักตามที่ออกแบบไว้

5.2 ข้อเสนอแนะและแนวทางพัฒนาต่อ

- เพิ่มการยืนยันตัวตนผู้ใช้งาน (Authentication/Authorization) แยกสิทธิการเข้าถึงตามตำแหน่งงาน เช่น เภสัชกร พยาบาล เจ้าหน้าที่การเงิน
- เพิ่ม CHECK Constraint และ Trigger เพื่อควบคุมความถูกต้องของข้อมูลในระดับฐานข้อมูลให้เข้มงวดยิ่งขึ้น
- ทำระบบรายงานสรุปเชิงสถิติ (Dashboard) เช่น รายได้ต่อเดือน ยาที่ถูกเบิกบ่อยที่สุด
- เพิ่มการสำรองข้อมูล (Backup) อัตโนมัติตามรอบเวลาที่กำหนด